

The Serotonin Operating System Manual

Volume 1: Kernel Architecture and Usermode Basics

Version: 0.4.0

February 25, 2026

Contents

1	Preamble	3
1.1	Open source acknowledgements	3
2	Hardware Support	8
2.1	Minimum Requirements	8
2.2	Video	8
2.3	Audio	8
2.3.1	PC Speaker	8
2.3.2	Yamaha YM3812 (OPL2)	9
2.3.3	Sound Blaster 16	9
2.4	Storage	9
2.4.1	IDE	9
2.4.2	Filesystems	9
3	Compilation	9
3.1	Application Binary Interface	9
3.2	GCC Cross Compilation	9
4	Kernel	9
4.1	Overview	9
4.2	Memory Management	10
4.3	Process, Signals, and IPC	10
4.4	Filesystems	10

4.5	Device and Timekeeping	10
5	The Serotonin Scheduler	11
5.1	Brief Overview	11
5.2	Process entry point	11
5.3	Process stack	11
5.4	Creating a kernel mode process	11
5.5	Creating a user mode process from kernel mode	12
5.6	Creating a user mode process from user mode	13
6	System Call Interface	13
6.1	Brief Overview	13
6.2	Error Handling	13
6.3	System Call List	13
6.3.1	Process and memory	13
6.3.2	Files, directories, and terminals	14
6.3.3	5HT compositor and system services	15
6.3.4	Reserved / not implemented	15
7	Kernel Compositor	15
7.1	Brief Overview	15
7.2	Compositing in kernel tasks	15
7.3	Compositing in user tasks	15
8	Usermode C Library	15
9	Usermode C++ Library	15
10	User Interface (Not Currently Implemented)	16
10.1	Taskbar	16
10.2	Window Layout	16
10.2.1	Tiling Window Manager - Ionic	16
10.2.2	Ionic Shortcuts	16
10.2.3	Floating / Stacking Window Manager - Covalence	16
10.3	Running Programs	17
10.3.1	Quick Run	17
10.3.2	Searching Programs	17
10.4	Settings	17

1 Preamble

This is a manual and is intended to supplement documentation. This manual will not explain every single function available in the operating system, you should also consult documentation for writing programs for Serotonin.

1.1 Open source acknowledgements

Serotonin uses the following open source projects:

- multiboot.h - Copyright (C) 1999,2003,2007,2008,2009,2010 Free Software Foundation, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL ANY DEVELOPER OR DISTRIBUTOR BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

- GNU Unifont - Copyright (C) GNU Project
GNU GENERAL PUBLIC LICENSE Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Lesser General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

GNU GENERAL PUBLIC LICENSE TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

a) You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.

b) You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.

c) If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print

or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

- a) Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
- b) Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
- c) Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or dis-

tribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.> Copyright (C)
<year> <name of author>

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

Gnomovision version 69, Copyright (C) year name of author Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type 'show w'. This is free software, and you are welcome to redistribute it under certain conditions; type 'show c' for details.

The hypothetical commands 'show w' and 'show c' should show the appropriate parts of the General Public License. Of course, the commands you use may be called something other than 'show w' and 'show c'; they could even be mouse-clicks or menu items—whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names:

Yoyodyne, Inc., hereby disclaims all copyright interest in the program 'Gnomovision' (which makes passes at compilers) written by James Hacker.

<signature of Ty Coon>, 1 April 1989 Ty Coon, President of Vice

This General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License.

2 Hardware Support

2.1 Minimum Requirements

The following specifications are required to run Serotonin:

- x86 CPU with Streaming SIMD Extensions 2 (SSE2) instruction set extension (Intel Pentium 4, AMD Athlon64, VIA C7 or newer)
- $\geq 1\text{GiB}$ of RAM
- Legacy BIOS with VESA BIOS Extensions 2.0 support
- $\geq \text{TBDGiB}$ of hard disk space

2.2 Video

Serotonin supports the VESA BIOS Extensions, preconfigured for a resolution of 1280x800 with 32-bit true colour (16,777,216 possible colours, excluding the alpha channel).

2.3 Audio

2.3.1 PC Speaker

The PC Speaker is capable of generating basic square waves.

2.3.2 Yamaha YM3812 (OPL2)

The YM3812 is a sound integrated circuit, popularised by the AdLib and Sound Blaster sound cards. It is capable of 9 melodic voice polyphony (6 melodic + 5 percussion in rhythm mode). Each voice has 2 operators for Frequency Modulation (FM) and additive synthesis, 4 selectable waveforms per operator (sine, half sine, absolute sine, quarters ine) and per operator 4-bit ADSR (attack, decay, sustain, release).

2.3.3 Sound Blaster 16

TBD

2.4 Storage

2.4.1 IDE

Serotonin currently only has support for legacy IDE ATA hard disks.

2.4.2 Filesystems

The following filesystems are supported:

- FAT32
- TMPFS/RAMFS

3 Compilation

3.1 Application Binary Interface

Serotonin uses the System V ABI, this includes the calling convention and executable format (ELF). Therefore, it is recommended to use the GNU Compiler Collection (GCC) when compiling both the kernel and and usermode programs.

3.2 GCC Cross Compilation

You should use the i686-elf target for cross compiling from a different operating system other then Serotonin. This will be subject to change when an OS specific toolchain is completed at some point in the future.

4 Kernel

4.1 Overview

The Serotonin kernel is a higher-half, monolithic kernel with preemptive usermode scheduling, a virtual filesystem, and a global compositor. It uses a per-task address space, a buddy allocator for physical memory, and a syscall ABI via `int 0x80`.

4.2 Memory Management

Serotonin uses paging with a higher-half kernel layout. Key virtual address ranges are:

- Kernel virtual base: 0xC0000000 (256 MiB mapped)
- Kernel heap: 0xD0000000 (256 MiB)
- Framebuffer: 0xE0000000
- Layer window region: 0xE1000000 up to 0xF0000000
- Kernel stack region: 0xF0000000 (4 MiB reserved)

User address space ranges:

- User code/data: 0x00400000 - 0xBFFFFFFF
- User heap: 0x20000000 - 0x5FFFFFFF
- Shared memory: 0x60000000 - 0x9FFFFFFF
- User stack: top at 0xBF000000, size 1 MiB, grows downward
- Signal trampoline: 0xBFF00000

Physical memory is managed by a buddy allocator with multiple zones, and per-process address spaces are switched on context switches.

4.3 Process, Signals, and IPC

Each task has a process control block (PCB) with register state, FPU state, file descriptor table, and scheduling metadata. The kernel supports:

- Preemptive usermode and cooperative kernel mode scheduling
- Signal delivery with 16 signal types and per-process handlers
- Shared memory objects (create/map) in a dedicated virtual range
- Kernel locks and semaphores for synchronization

4.4 Filesystems

Serotonin provides a virtual filesystem (VFS) with pluggable filesystem drivers. Current drivers include FAT32 and TMPFS/RAMFS. Processes track a current working directory and operate on files using file descriptors backed by VFS nodes.

4.5 Device and Timekeeping

The kernel includes drivers for PIT-based time slicing, RTC time-of-day, PS/2 keyboard and mouse, serial output, legacy IDE storage, VBE graphics, and basic audio output (PC speaker and OPL2).

5 The Serotonin Scheduler

5.1 Brief Overview

The Serotonin Scheduler is a priority round-robin scheduler for kernel mode (Ring 0, "Supervisor mode") and user mode processes (Ring 3). Kernel mode tasks are cooperative (that is, you must explicitly yield or block in your code), and usermode tasks are preemptive (the scheduler will preempt your process on the PIT timer). The time quantum is 10ms (`SCHEDULE_QUANTUM`) and priority decay is applied after consuming quanta. There are 256 possible priorities, 0 being the lowest (idle task) and 255 the highest (`/bin/init`). The scheduler always picks the highest priority task that is ready to run; tasks of equal priority are scheduled round-robin.

5.2 Process entry point

You must provide the entry point of the process. The scheduler will then start execution from the provided pointer, provided that the process is in the scheduler queue, the process is ready and is not blocked. Kernel mode code runs in the higher-half mapping at `0xC0000000+`; user mode binaries are expected to load into user space (`0x00400000 - 0xBFFFFFFF`).

5.3 Process stack

The scheduler will allocate a stack for you, the size and location depends on the privilege level. Kernel mode stacks are 16 KiB each, allocated from the top of the `0xF0000000 - 0xF03FFFFFFF` region and grow downward. User mode stacks are 1 MiB each, allocated from `0xBF000000` downward and grow downward.

5.4 Creating a kernel mode process

You can create a new kernel mode process by creating a process and adding it to the scheduler queue:

```

1  #include "stdio/stdio.h"
2  #include "schedule/schedule.h"
3
4  void my_process(void)
5  {
6      for (int i = 0; i < 5; i++) {
7          printf("Hello, kernel world!\n");
8          kernel_yield();
9      }
10     task_exit(get_current_task(), EXIT_SIGTERM);
11 }
12
13 // kernel_main_high
14 process_control_block_t *my_pcb = task_create(my_process, "my_process",
15     CPU_KERNEL_MODE, 128);
16 enqueue(my_pcb);

```

Note that you have to yield the CPU after you are done with your computation. This is important so that other processes can have CPU time and are not starved. Failing to yield the CPU can result in lockups/freezes, and this is not ideal for the end user experience.

5.5 Creating a user mode process from kernel mode

While in most cases, this is not required, as you should be creating usermode tasks from usermode, it is possible to do so by loading an ELF binary.

```

1 # /bin/myusermodetask
2 .section .rodata
3 msg: .asciz "Hello, world!\n"
4
5 .section .text
6 .global _start
7 _start:
8     movl $1,    %eax # WRITE
9     movl $0,    %ebx # STDOUT
10    movl $msg, %ecx # pointer to message
11    movl $14, %edx # length
12    int $0x80      # send syscall to kernel
13
14    movl $0,    %eax # EXIT
15    movl $15, %ebx # SIGTERM
16    int $0x80      # send syscall to kernel

```

```

1 #include "stdio/stdio.h"
2 #include "schedule/schedule.h"
3 #include "kernel.h"
4
5 // kernel_main_high
6 char* elf_loc = "/bin/mytask";
7 process_control_block_t *init = task_create(NULL, elf_loc,
8     CPU_USER_MODE, 128);
9 const char *argv[1] = {"/bin/mytask"}; int argc = 1;
10 const char *envp[1] = {"PATH=/"}; int envc = 1;
11 int init_status = kernel_load_elf(init, elf_loc, elf_loc, argv, argc,
12     envp, envc);
13 kernel_yield();

```

The `kernel_load_elf` function will automatically enqueue the task into the scheduler. You can now yield from the current kernel task. Notice how there is no "yield" in the usermode task. This is because all usermode tasks are preempted by the kernel. You can read more about the Serotonin system call interface in the "System Call Interface" chapter.

5.6 Creating a user mode process from user mode

You can create a new user mode process by using the fork and execve system calls. More detail can be viewed in the "System Call Interface" chapter.

```

1  .section .text
2  .global _start
3  _start:
4      movl $4,    %eax # FORK
5      int  $0x80      # send syscall to kernel
6
7      cmpl $0,    %eax
8      je   child      # eax==0 if child
9      jg   parent      # eax==child_pid if parent (eax>0)
10
11     jmp  error        # eax<0 if fork failed
12
13 child:
14     # do whatever in child
15
16 parent:
17     # do whatever in parent
18
19 error:
20     # handle error

```

6 System Call Interface

6.1 Brief Overview

Serotonin uses the interrupt 0x80 to handle system calls generated by usermode tasks, and arguments are taken from the general purpose registers eax, ebx, ecx and edx. The eax register is used to specify the system call operation, while ebx, ecx and edx are used as arguments for the system call. For example, the exit system call uses eax=0, and ebx is used to specify the exit code.

6.2 Error Handling

System calls return their result in eax. On failure, the kernel returns a negative errno value in eax. User-space wrappers translate this into `errno`.

6.3 System Call List

6.3.1 Process and memory

- 0 EXIT (ebx=status) terminate the current process.
- 3 EXECVE (ebx=path, ecx=argv, edx=envp) replace the current process image.
- 4 FORK duplicate the current process.
- 5 GETPID return the current process ID.

- 9 WAITPID (ebx=pid, ecx=status*) wait on a child.
- 11 SBRK (ebx=incr) adjust the program break.
- 17 KILL (ebx=pid, ecx=sig) send a signal.
- 18 SIGNAL (ebx=sig, ecx=handler) register a signal handler.
- 19 SIGRET return from a signal handler.
- 25 PAUSE block until a signal is received.
- 26 SHM_CREATE (ebx=size) create a shared memory object.
- 27 SHM_MAP (ebx=shm_id) map a shared memory object.
- 28 SHM_UNMAP (ebx=addr) unmap a shared memory object (currently returns ENOSYS).
- 38 PIPE (ebx=fds) creates read/write pipes given an array of file descriptors.

6.3.2 Files, directories, and terminals

- 1 WRITE (ebx=fd, ecx=buf, edx=count) write to a file descriptor.
- 2 READ (ebx=fd, ecx=buf, edx=count) read from a file descriptor.
- 6 TTY (ebx=fd) test if a file descriptor is a TTY.
- 7 OPEN (ebx=path, ecx=flags) open a file.
- 8 CLOSE (ebx=fd) close a file descriptor.
- 10 TOD (ebx=tv, ecx=tz) read RTC time (tz unused).
- 14 LSEEK (ebx=fd, ecx=offset, edx=whence) seek within a file.
- 15 STAT (ebx=path, ecx=stat*) stat a path.
- 16 FSTAT (ebx=fd, ecx=stat*) stat a file descriptor.
- 20 MKDIR (ebx=path) create a directory.
- 21 RMDIR (ebx=path) remove an empty directory.
- 22 CHDIR (ebx=path) change the current working directory.
- 23 GETCWD (ebx=buf, ecx=size) get the current working directory.
- 24 UNLINK (ebx=path) delete a file.
- 30 LISTDIR (ebx=path, ecx=buf, edx=size) return a newline-delimited listing.
- 37 DUP (ebx=oldfd, ecx=-1) duplicates a file descriptor to the next available file descriptor.
- 37 DUP2 (ebx=oldfd, ecx=newfd) duplicates a file descriptor to a new file descriptor.

6.3.3 5HT compositor and system services

- 29 5HT_LIST_PROC (ebx=buf, ecx=max) list running processes.
- 31 5HT_REQ_BUF (ebx=layer_id, ecx=cfg*, edx=out*) allocate a framebuffer layer.
- 32 5HT_REL_BUF (ebx=layer_id) release a framebuffer layer.
- 33 5HT_RCFG_LAYER (ebx=layer_id, ecx=cfg*, edx=out*) reconfigure a layer.
- 34 5HT_QUERY_INFO (ebx=out*) query global framebuffer info.
- 35 5HT_QUERY_LAYER (ebx=layer_id, ecx=out*) query a layer.
- 36 5HT_SET_FID (ebx=pid) set the foreground input task.

6.3.4 Reserved / not implemented

- 12 ENVIRON (reserved)
- 13 LINK (reserved)

7 Kernel Compositor

7.1 Brief Overview

The Serotonin kernel provides a global compositor for all tasks, kernel and user. The compositor manages 16 z layers and supports alpha blending for all layers except the base layer (layer zero). User tasks can allocate and configure layers through the 5HT system calls, writing pixel data into shared layer buffers and signaling frame completion through layer metadata.

7.2 Compositing in kernel tasks

7.3 Compositing in user tasks

8 Usermode C Library

Serotonin user programs use a newlib-based C library with syscall wrappers for kernel services. Startup is provided by `crt0.o`, and the syscall glue in `user/syscall/syscall.c` exposes POSIX-like APIs (stdio, file I/O, process control, signals, and basic filesystem operations). Memory allocation uses `sbrk` backed by the kernel user heap. The library is intentionally small and does not provide full POSIX coverage yet.

9 Usermode C++ Library

Serotonin provides a minimal C++ runtime and STL support for userland. The runtime supplies global constructor/destructor handling (`cxx_init.o`), `__cxa_atexit`/`__cxa_finalize`, `__cxa_pure_virtual`, and `new/delete` operators. The standard library uses STLport 5.2.1 with stubs for exception-related throws; builds are configured without C++ exceptions and error paths abort instead of throwing. `iostream` and basic STL containers are supported, but higher-level facilities may be incomplete.

10 User Interface (Not Currently Implemented)

Anything stated is planned, but subject to change.

10.1 Taskbar

Serotonin's UI contains a simple taskbar with a list of active programs, system information like CPU, GPU, RAM and Storage usage. The taskbar location can be changed in the settings, alongside its width and the colour.

10.2 Window Layout

10.2.1 Tiling Window Manager - Ionic

Serotonin implements a tiling window manager (WM) called Ionic, named after Ionic bonds. The WM is a simplistic, low fidelity UI made purely for performance. This is advised for lower end hardware nearer to the Hardware Requirements spec. Windows are organised in a grid-like structure. This WM is enabled by default in the settings.

Ionic features 2-dimensional workspaces, meaning users can have multiple virtual desktops arranged both horizontally and vertically, categorized with numbers and letters.

10.2.2 Ionic Shortcuts

Action	Shortcut
Move window	MOD + ARROW_KEY
Swap workspace (horizontal)	MOD + CTRL + ARROW_KEY
Move window between workspaces (horizontal)	MOD + SHIFT + ARROW_KEY
Swap workspace (vertical)	MOD + CTRL + PAGE UP/PAGE DOWN
Move window between workspaces (vertical)	MOD + SHIFT + PAGE UP/PAGE DOWN
Move window to workspace 1-9	MOD + SHIFT + NUM
Move window to workspace Q-E	MOD + SHIFT + LETTER

Table 1: Ionic Window Manager Shortcuts

10.2.3 Floating / Stacking Window Manager - Covalence

Covalence is a floating WM implemented into Serotonin as an alternative to Ionic. Covalence is much more intuitive for end users familiar with operating systems like MacOS, Windows and other WMs like Gnome or KDE. Named after covalence bonds, this window manager allows users to move, resize, minimize and close windows with their mouse, or optionally with keyboard shortcuts. Naturally this WM is more computationally expensive and must be enabled manually by the user.

10.3 Running Programs

10.3.1 Quick Run

Processes can be opened using the default shortcut `MOD + D` which opens the program selection dialog, [insert name here]. The user can then open a previous task using `CTRL + NUM`, where `NUM` is a number key from 1 to 3.

10.3.2 Searching Programs

The user can also begin to type the name of a program to search the system for it instead. The program is then run by pressing `ENTER`. The arrow keys or tab can be used to cycle through the current options, as well as double clicking the program in the list.

10.4 Settings

Serotonin allows users to customize their desktop experience inside the settings program. Users can change their wallpaper, taskbar colour and location, default window manager and other settings as they get implemented. The menu will render using an Ionic-style window so low end hardware can always run the settings app. This guarantees they can configure their system to perform well.